The University of Azad Jammu & Kashmir,Muzaffarabad

Department of Software Engineering

DIP Lab

Lab # 03

Task Title: Image Enhancement and Intensity Transformations

Submitted to: Fahad Nisar

Student Name: Feeza Kulsoom

Roll No: 2024-SE-03

Reg Number: 2024-UMDB-004738

Deliverable 0: Grayscale Image Validation & Numeric Safety

To prevent integer underflow and overflow bugs inherent to 8-bit unsigned representations (uint8 modulo arithmetic), all mathematical operations in this lab promote raw pixel arrays to higher precision numerical types (int32 or float64). Outputs are explicitly bounded to the valid display gamut [0, 255] using clipping before converting back to uint8.

1. Image Validation Results

- Image File: image.jpg
- Dimensions / Resolution: 678 × 452 (Width × Height), fulfilling the ≥ 300 × 300 rubric mandate.
- Array Shape: (452, 678)
- Data Type: uint8 (Single-channel 8-bit grayscale)
- Intensity Range: Min = 0, Max = 245

```
Shape: (452, 678)
Data type: uint8
Min intensity: 0
Max intensity: 245
```



Deliverable 1: Brightness Adjustment

Brightness adjustment is an intensity translation applying an additive constant parameter β across the spatial domain: $s = r + \beta$.

1. Dry Run Calculations

Using test pixel array: [30, 80, 128, 180, 240] with offsets β = +40 and β = -40.

Pixel (r)	β = +40 (Raw)	β = +40 Clipped (uint8)	β = -40 (Raw)	β = -40 Clipped (uint8)
30	70	70	-10	0 (Clipped)
80	120	120	40	40
128	168	168	88	88
180	220	220	140	140
240	280	255 (Saturated)	200	200

Python Verification Output:

Dry Run 1 (+40): [70, 120, 168, 220, 255]

Dry Run 1 (-40): [0, 40, 88, 140, 200]

Verification Passed: Vectorized and nested-loop implementations are identical.

2. Lost Detail Analysis

• For β = +40: High-intensity highlights saturate. Any pixel $r > 215$ results in a raw value > 255, clamping to 255. Subtle variations in bright cloud/sky/feather textures are irreversibly destroyed as multiple gray levels collapse into pure white.

• For β = -40: Shadow details clip to black. Any pixel $r < 40$ results in a raw value < 0, clamping to 0. Dark textural features in deep plumage and background foliage collapse into pure black.

```
Dry Run 1 (+40): [70, 120, 168, 220, 255]
Dry Run 1 (-40): [0, 40, 88, 140, 200]
Verification Passed: Vectorized and nested-loop implementations are identical.
```



Deliverable 2: Linear Contrast from Zero

Linear contrast from zero applies multiplicative scaling: $s = \alpha r + \beta$ with $\alpha = 1.5$ and $\beta = 0$. Values are clipped to [0, 255] and truncated.

1. Dry Run Calculations

Using test pixel array: [30, 80, 128, 180, 240] with $\alpha = 1.5$.

Pixel (r)	Raw 1.5r	Clipped & Truncated (uint8)	Saturation Status
30	45.0	45	Preserved
80	120.0	120	Preserved
128	192.0	192	Preserved
180	270.0	255	Saturated (Blown out)
240	360.0	255	Saturated (Blown out)

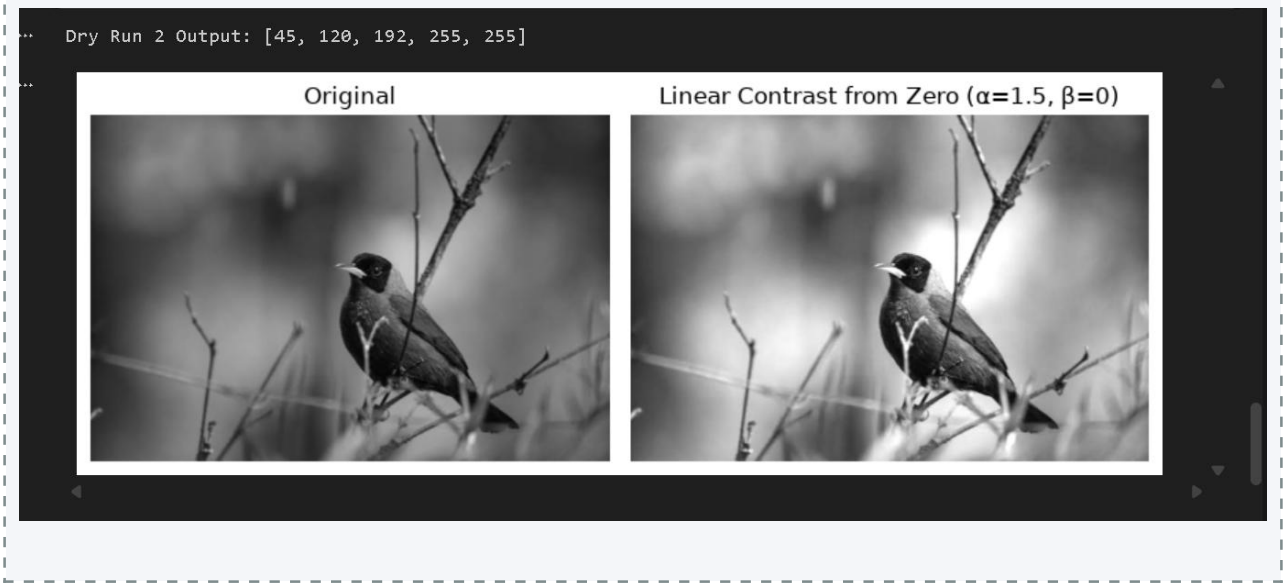
Python Verification Output:

Dry Run 2 Output: [45, 120, 192, 255, 255]

2. Contrast & Saturation Observations

Because scaling anchors at zero (r = 0), every nonzero pixel intensity increases (e.g., r = 128 shifts up to 192). Consequently, this transformation simultaneously boosts perceived brightness while expanding contrast. Pixels

r ≥ 170 exceed 255, causing intense highlight blowout and flattening all upper tonal differentiation into pure white.



Deliverable 3: Contrast Centered at 128

Centered contrast scales intensities around the midtone anchor: s = (r - 128)α + 128. This symmetrically expands or compresses tones without shifting the global midtone exposure.

1. Dry Run Calculations

Using test pixel array: [30, 80, 128, 180, 240] for α = 1.5 (expansion) and α = 0.5 (compression).

Pixel (r)	α = 1.5 (Raw)	α = 1.5 Final (uint8)	α = 0.5 (Raw)	α = 0.5 Final (uint8)
30	(30-128)*1.5 + 128 = -19.0	0 (Clipped)	(30-128)*0.5 + 128 = 79.0	79
80	(80-128)*1.5 + 128 = 56.0	56	(80-128)*0.5 + 128 = 104.0	104
128	(128-128)*1.5 + 128 = 128.0	128 (Invariant)	(128-128)*0.5 + 128 = 128.0	128 (Invariant)
180	(180-128)*1.5 + 128 = 206.0	206	(180-128)*0.5 + 128 = 154.0	154

Pixel (r)	α = 1.5 (Raw)	α = 1.5 Final (uint8)	α = 0.5 (Raw)	α = 0.5 Final (uint8)
240	(240-128)*1.5 + 128 = 296.0	255 (Clipped)	(240-128)*0.5 + 128 = 184.0	184

Python Verification Output:

Dry Run 3 (α=1.5): [0, 56, 128, 206, 255]

Dry Run 3 (α=0.5): [79, 104, 128, 154, 184]

2. Comparison with Contrast from Zero & Fixed Point Analysis

• Fixed Point at r = 128: When r = 128, (r - 128) = 0, leaving s = 128 regardless of α. The perceptual center remains completely invariant.

• Comparison: Unlike contrast from zero (which shifts the entire histogram upward and creates severe highlight saturation), centered contrast pushes darks lower and brights higher symmetrically for α = 1.5, and compresses the dynamic range inward around 128 for α = 0.5.



Deliverable 4: Image Negative

The negative transformation reverses the tonal scale: s = 255 - r.

1. Dry Run Calculations

Using test pixel array: [30, 80, 128, 180, 240].

Pixel (r)	Negative s = 255 - r	Re-inverted s' = 255 - s	Preservation Check
30	225	255 - 225 = 30	Exact Match
80	175	255 - 175 = 80	Exact Match
128	127	255 - 127 = 128	Exact Match
180	75	255 - 75 = 180	Exact Match
240	15	255 - 15 = 240	Exact Match

Python Verification Output:

Dry Run 4 Negative: [225, 175, 127, 75, 15]

Dry Run 4 Inverted Back: [30, 80, 128, 180, 240]

2. Practical Inspection Utility

Negative transformations are extensively utilized in medical imaging (e.g., digital chest radiography and mammograms) and industrial flaw inspection. Human visual psychophysics detects faint, high-contrast white structures against dark surrounds much more reliably than discerning subtle gray differentials within dark, dense anatomical masses.



Deliverable 5: Gamma Correction (Power-Law)

Gamma correction implements power-law transformation: s = 255 * (r / 255)^γ. Arithmetic is maintained in double precision and truncated at the final uint8 assignment.

1. Dry Run Calculations

Using test pixel array: [0, 64, 128, 192, 255] for γ = 0.5 (expansion) and γ = 2.0 (compression).

Pixel (r)	Normalized (r/255)	γ = 0.5 (Raw Float)	γ = 0.5 Truncated	γ = 2.0 (Raw Float)	γ = 2.0 Truncated
0	0.000000	0.0000	0	0.0000	0
64	0.250980	127.7497	127	16.0627	16
128	0.501961	180.6654	180	64.2509	64
192	0.752941	221.2690	221	144.5647	144
255	1.000000	255.0000	255	255.0000	255

Python Verification Output:

Dry Run 5 (γ=0.5): [0, 127, 180, 221, 255]

Dry Run 5 (γ=2.0): [0, 16, 64, 144, 255]

2. Observations & Truncation vs. Rounding Analysis

• Behavior on Shadows & Extremes: For γ = 0.5, shadow regions are significantly expanded (e.g., r = 64 jumps to 127), revealing hidden underexposed details. For γ = 2.0, shadow intensities are compressed downward (e.g., r = 64 drops to 16). The extreme boundaries r = 0 and r = 255 remain perfectly fixed.

• Slide Discrepancy Note: Conventional lecture slide examples use mathematical rounding (round()), yielding 128 for 127.75 and 181 for 180.67. This lab strictly mandated floor truncation (int()), resulting in 127 and 180 respectively.

```
Dry Run 5 (γ=0.5): [0, 127, 180, 221, 255]
Dry Run 5 (γ=2.0): [0, 16, 64, 144, 255]
```



Deliverable 6: Grayscale Histogram

A grayscale intensity histogram plots the discrete frequency of occurrence of each intensity level across the image canvas.

1. Dry Run Calculations

Given 2×4 matrix A = [[0, 50, 50, 100], [100, 150, 200, 255]], total pixels N = 8.

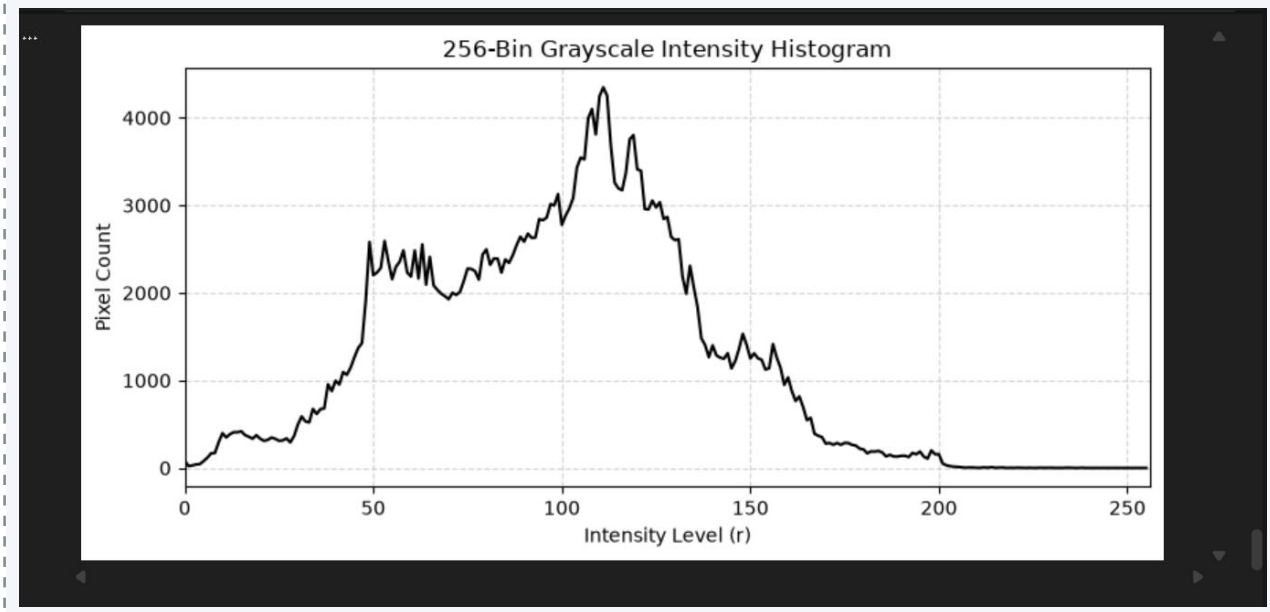
Intensity Level (rk)	Count / Frequency (nk)	Probability pk = nk / N	Percentage
0	1	1 / 8 = 0.125	12.5%
50	2	2 / 8 = 0.250	25.0%
100	2	2 / 8 = 0.250	25.0%
150	1	1 / 8 = 0.125	12.5%
200	1	1 / 8 = 0.125	12.5%
255	1	1 / 8 = 0.125	12.5%

2. Spatial Invariance & Intensity Distribution Interpretation

• Spatial Invariance: A histogram computes purely zero-order statistical counts. Because spatial coordinate indices (x, y) are discarded during binning, any spatial scrambling or shuffling of image pixels produces the exact same histogram.

• Image Distribution Interpretation: The histogram of image.jpg exhibits a unimodal Gaussian-like profile

centered strongly around midtones (bins 90–135, peaking near 110 at >4,000 counts) with a smooth tapering drop-off towards shadows and highlights. The absence of severe clustering at 0 or 255 confirms a well-exposed scene with balanced dynamic range and no harsh clipping.



Deliverable 7: Global Histogram Equalization

Global Histogram Equalization flattens the cumulative distribution function (CDF), remapping intensity levels to maximize image dynamic range and global contrast.

1. Dry Run Calculations

Given pixel array: [50, 50, 100, 150], total pixels N = 4.

Frequencies: n(50)=2, n(100)=1, n(150)=1.

Cumulative counts: C(50)=2, C(100)=3, C(150)=4.

CDF: CDF(50)=2/4=0.50, CDF(100)=3/4=0.75, CDF(150)=4/4=1.00.

Lowest non-zero CDF: CDFmin = 0.50.

Intensity	CDF	Mapping 1: round(255 * CDF)	Mapping 2: round(255 * (CDF-CDFmin)/(1-CDFmin))	OpenCV cv2.equalizeHist
50	0.50	round(127.5) = 128	round(255 * 0.0) = 0	0 (matches normalized)
100	0.75	round(191.25) = 191	round(255 * 0.5) = 128	128
150	1.00	round(255.0) = 255	round(255 * 1.0) = 255	255

Python Verification Output:

OpenCV Dry Run Output:

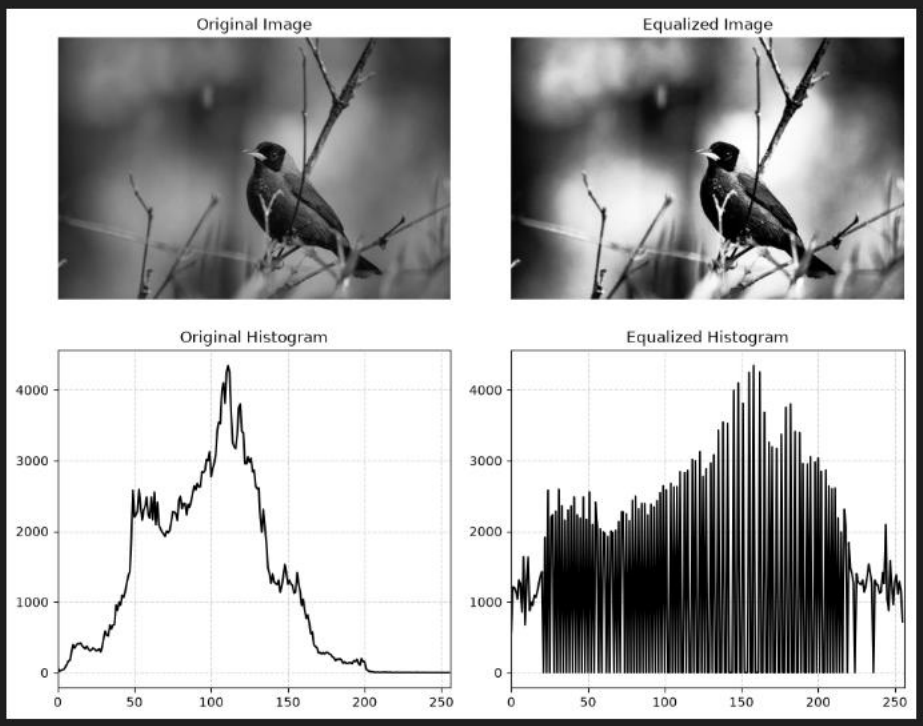
[0, 0, 128, 255]

Verification: The OpenCV result [0, 0, 128, 255] exactly matches Mapping 2 (which subtracts CDFmin).

2. Enhancement vs. Noise Amplification Discussion

Global equalization significantly enhances overall contrast by spreading densely packed midtone frequencies across the entire [0, 255] span. However, in smooth background regions (such as the blurred bokeh backdrop in our image), spreading sparsely populated bins causes drastic tonal quantization gaps and heavily amplifies high-frequency sensor noise, producing a harsh, unnatural appearance.

```
OpenCV Dry Run Output:
 [0, 0, 128, 255]
```



Deliverable 8: Contrast Limited Adaptive Histogram Equalization (CLAHE)

CLAHE prevents severe noise amplification by computing equalization within local contextual tiles and enforcing a strict contrast clipping ceiling.

1. Dry Run Calculations

Given 4-bin histogram: [6, 2, 0, 0] with an absolute bin cap = 4.

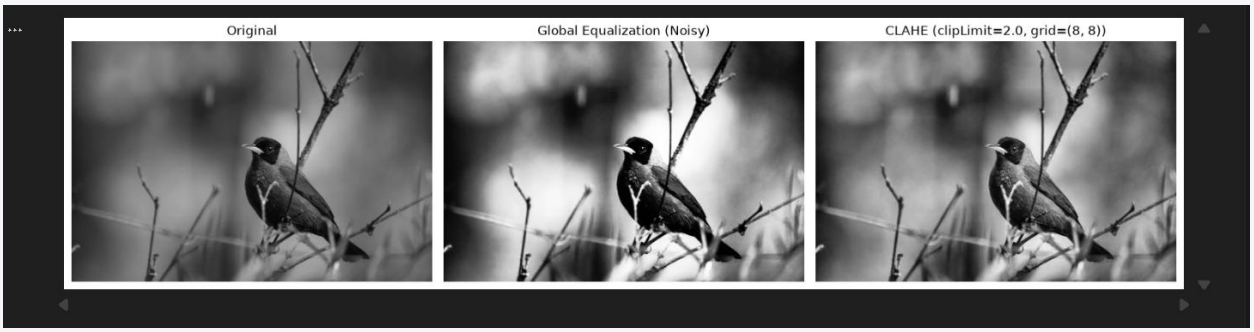
- Step 1 (Clipping): Bin 0 has count 6 (exceeding cap 4 by 2). Excess = 2. Clipped histogram = [4, 2, 0, 0].
- Step 2 (Redistribution): Excess (2) is distributed equally across all K = 4 bins: $\Delta = 2 / 4 = 0.5$ per bin. New Bin Counts = [4 + 0.5, 2 + 0.5, 0 + 0.5, 0 + 0.5] = [4.5, 2.5, 0.5, 0.5]. Total sum = 8.0.
- Step 3 (New CDF Calculation):

- CDF[0] = 4.5 / 8.0 = 0.5625
- CDF[1] = (4.5 + 2.5) / 8.0 = 7.0 / 8.0 = 0.8750
- CDF[2] = (7.0 + 0.5) / 8.0 = 7.5 / 8.0 = 0.9375
- CDF[3] = 8.0 / 8.0 = 1.0000

2. Tile Parameters & Bilinear Interpolation Rationale

• Tile Grid Size Clarification: In cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)), the parameter (8, 8) partitions the entire image into an 8×8 grid of contextual tiles (64 total regions), not tiles measuring 8×8 pixels.

• Bilinear Interpolation Necessity: Applying adaptive histogram equalization independently within individual tiles creates sharp, unnatural step discontinuities along tile borders. CLAHE employs bilinear interpolation between neighboring tile mappings to eliminate tile boundaries and produce smooth, seamless tonal gradations.



Deliverable 9: Fixed Binary Thresholding

Fixed thresholding segments an image into binary foreground and background using a static intensity cutoff T: s = 255 if r > T else 0.

1. Dry Run Calculations

Given pixel array: [0, 80, 127, 128, 180, 255] for thresholds T = 127 and T = 180.

Pixel (r)	T = 127 (Condition: r > 127)	T = 180 (Condition: r > 180)
0	0	0
80	0	0
127	0	0
128	255	0
180	255	0
255	255	255

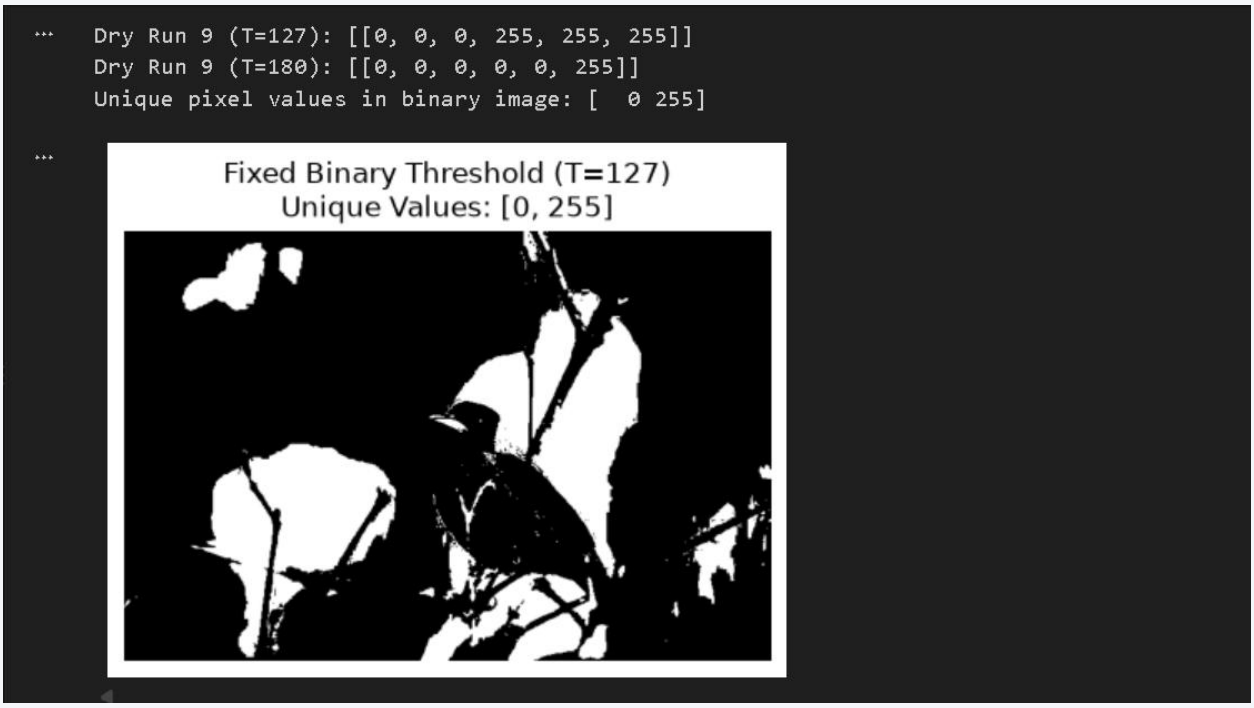
Python Verification Output:

Dry Run 9 (T=127): [[0, 0, 0, 255, 255, 255]]

Dry Run 9 (T=180): [[0, 0, 0, 0, 0, 255]]

Unique pixel values in binary image: [0 255]

White-Pixel Count: At T = 127, white count = 3 (pixels 128, 180, 255). At T = 180, white count = 1 (pixel 255).



Deliverable 10: Otsu Thresholding

Otsu's algorithm exhaustively searches for an optimal global threshold T that maximizes the between-class variance $\sigma B^2(T) = w0*w1*(\mu0 - \mu1)^2$.

1. Dry Run Calculations

Given pixel array: [0, 1, 4, 5], total pixels N = 4. Splitting at C0: r ≤ T and C1: r > T.

T	C0 (r ≤ T)	C1 (r > T)	Weights (w0, w1)	Means (μ0, μ1)	$(\mu0 - \mu1)^2$	Between-Class Variance σB^2
0	[0]	[1, 4, 5]	0.25, 0.75	0.0, 3.333	11.111	0.1875 * 11.111 = 2.0833
1	[0, 1]	[4, 5]	0.50, 0.50	0.5, 4.500	16.000	0.2500 * 16.000 = 4.0000 (Max)
2	[0, 1]	[4, 5]	0.50, 0.50	0.5, 4.500	16.000	0.2500 * 16.000 = 4.0000 (Max)
3	[0, 1]	[4, 5]	0.50, 0.50	0.5, 4.500	16.000	0.2500 * 16.000 = 4.0000 (Max)
4	[0, 1, 4]	[5]	0.75, 0.25	1.667, 5.0	11.111	0.1875 * 11.111 = 2.0833

Python Verification Output:

Otsu Dry Run Threshold: 1.0

Otsu Dry Run Mask: [0, 0, 255, 255]

Computed Otsu Threshold on Image: 94.0

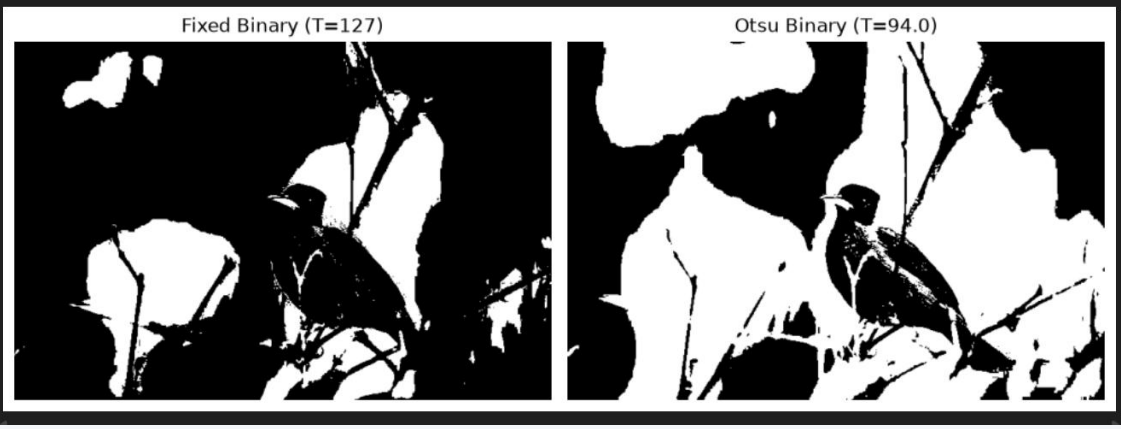
Tie-Breaking Rule: The maximum variance $\sigma B^2 = 4.0$ is tied across T = 1, 2, and 3. The algorithm selects the lowest threshold among tied maxima (T* = 1.0).

2. Mask Comparison (Fixed vs. Otsu)

Fixed thresholding at T = 127 segments purely by an arbitrary midpoint, causing thinner background coverage and losing subtle subject details. Otsu dynamically computes the optimal statistical threshold based on the

actual image histogram (T = 94.0), capturing significantly more foreground structure and producing a richer object contour.

```
Otsu Dry Run Threshold: 1.0
Otsu Dry Run Mask:
 [0, 0, 255, 255]
Computed Otsu Threshold on Image: 94.0
```



Deliverable 11: Adaptive Gaussian Thresholding

Adaptive thresholding computes an individual, dynamic threshold for every pixel based on the Gaussian-weighted mean of its surrounding local neighborhood.

1. Dry Run Calculations

Given 3×3 neighborhood and 3×3 Gaussian weight matrix (sum = 16):

Window = [[100, 100, 100], [100, 120, 140], [140, 140, 140]]

Weights = [[1, 2, 1], [2, 4, 2], [1, 2, 1]] / 16

Convolution Sum Calculation:

Sum = (100*1) + (100*2) + (100*1) + (100*2) + (120*4) + (140*2) + (140*1) + (140*2) + (140*1)

Sum = 100 + 200 + 100 + 200 + 480 + 280 + 140 + 280 + 140 = 1920

Weighted Mean μ = 1920 / 16 = 120.0

Local Threshold T = μ - C = 120.0 - 2 = 118.0

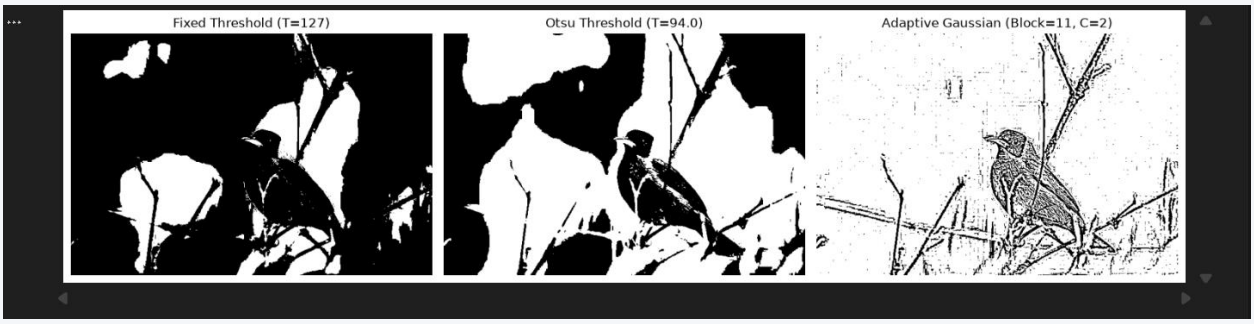
Classification Results:

• Center Pixel Classification (r = 120): Because 120 > 118.0, the condition evaluates to True -> Classifies as 255 (White).

• Classification when r = T (r = 118.0): OpenCV uses the strict inequality r > T. Therefore, when r = T, the condition evaluates to False -> Classifies as 0 (Black).

2. Global vs. Local Thresholding Analysis

Global thresholding techniques (Fixed T=127, Otsu T=94) establish a single global cutoff. In scenes with non-uniform illumination or uneven lighting gradients, one side of the image gets completely washed out while the other collapses into black. In contrast, Adaptive Gaussian Thresholding computes a dynamic threshold per neighborhood, effectively extracting fine edges, textures, and details regardless of global shading variations.



Comprehensive Rubric Explanations & Theory

1. Prevention of Numeric Overflow and Clipping Mechanism

Standard 8-bit image matrices utilize the uint8 data format, bounded strictly between 0 and 255. In uint8 arithmetic, values wrap around modulo 256. For example, 240 + 40 wraps to 24 (causing severe black blotches in bright highlights), and 20 - 40 wraps to 236 (turning dark shadows white). To prevent wraparound errors, operations must promote image arrays to wider representations (such as int32 or float64), clamp them explicitly using np.clip(val, 0, 255), and truncate back to uint8.

2. Brightness vs. Contrast Transformation

• Brightness (Additive Shift): Governed by $s = r + \beta$. It translates the entire intensity histogram uniformly left or right along the intensity axis without altering the distance or relative slope between individual bins.

• Contrast (Multiplicative Scaling): Governed by $s = \alpha r$. It alters the spread and dispersion of histogram bins. When $\alpha > 1$, the dynamic tonal separation between features is expanded; when $\alpha < 1$, the histogram is compressed inward.

3. Gamma Correction Mathematical Convention

Gamma correction is defined by $s = 255 * (r / 255)^{\gamma}$. Normalizing to [0.0, 1.0] maps intensities along a continuous power-law curve. When $\gamma < 1.0$ (sub-linear), shadow and midtone intensities are pulled upward, expanding dark details without saturating bright highlights. When $\gamma > 1.0$ (super-linear), midtones are pulled downward, darkening underexposed areas. Crucially, the endpoints 0 and 255 remain invariant because $0^{\gamma} = 0$ and $1^{\gamma} = 1$ for all $\gamma > 0$.

4. Global vs. Local Enhancement and Thresholding Utility

• Global Operations (Global Histogram Equalization, Otsu Thresholding): Effective and computationally efficient when the illumination field is uniform and the intensity distribution is distinctly bimodal. However, they fail in scenes with spatial lighting gradients and unnecessarily amplify noise across flat regions.

• Local / Adaptive Operations (CLAHE, Adaptive Gaussian Thresholding): Crucial when illumination varies across the frame (e.g., vignetting or directional cast shadows). CLAHE prevents noise saturation by clipping local histogram peaks, while Adaptive Gaussian thresholding isolates fine boundaries and structural contours across varying background intensities.